

Assignment 6: Windows, Tabs, and Iframes

Multi-Window Context Switching, OS-Level Window Handles, and Nested Frame Boundary Traversal

STUDENT & ACADEMIC WORK RECORD

Candidate Name	Souvik Nandi
Student ID / Enroll No.	12023002028046
Course/Track	Python for Automation
Curriculum Tier	Tier 2: Advanced User Interactions
Academic Session	Academic Year 2023-2027
Submission Date	11-Sep-2026

EXECUTIVE ABSTRACT

This laboratory report addresses execution context segregation in modern web applications. Modern web suites employ nested DOM documents (`<iframe>`) for component sandboxing and multi-tab workflows for third-party redirects. Because standard WebDriver locators are bound strictly to the active browsing context, queries across frame boundaries or secondary window tabs trigger immediate `NoSuchElementException` failures. By employing `driver.switch_to.frame()`, `driver.switch_to.default_content()`, and tracking window identifiers via `driver.window_handles`, this experiment demonstrates deterministic multi-context traversal, cross-document assertion, and clean process teardown.



Certified Academic Submission Compliant with Laboratory Work Record Specification Standards

TIER 2: ADVANCED USER INTERACTIONS • ASSIGNMENT 6

1. PROBLEM STATEMENT & ENGINEERING SCOPE

Enterprise web architectures frequently partition functionalities across distinct DOM tree boundaries (iframe containers) and external browser window handles. When an application spawns a new tab or nests a rich-text editor within an isolated frame, automated test engines fail to locate targeted components unless the execution context is explicitly transitioned. The technical mandate requires navigating to an embedded iframe, injecting text into a nested WYSIWYG editor, reverting cleanly to root context, triggering a new browser tab, switching to the newly spawned window handle to assert its metadata, and terminating the secondary tab without crashing the root session.

2. TECHNICAL OBJECTIVES & SUCCESS CRITERIA

- **Frame Boundary Traversal:** Transition WebDriver scope into an isolated iframe sandbox via `switch_to.frame` and return to the root DOM via `switch_to.default_content`.
- **Window Handle Resolution:** Capture unique 128-bit window handle hex strings, identify newly spawned tabs, and transition focus using `switch_to.window`.
- **Deterministic Teardown:** Isolate child tab destruction via `driver.close()` from complete session termination (`driver.quit()`).

3. CONTEXT SWITCHING ARCHITECTURE COMPARISON MATRIX

Target Context	Selenium Switch API	Scope Boundary	Reset Command
Embedded iFrame	<code>driver.switch_to.frame(id)</code>	Sub-document DOM isolated by frame boundary	<code>driver.switch_to.default_content()</code>
Browser Window / Tab	<code>driver.switch_to.window(handle)</code>	Separate OS-level rendering process/tab	<code>driver.switch_to.window(main_h)</code>
Parent Frame	<code>driver.switch_to.parent_frame()</code>	Immediate parent container in nested frames	Traverse up one step in frame stack

4. SOURCE CODE IMPLEMENTATION (PYTHON SELENIUM)

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support import WebDriverWait from selenium.webdriver.support import expected_conditions as EC # Initialize Chrome WebDriver session driver = webdriver.Chrome() try: wait = WebDriverWait(driver, timeout=10) # ===== PHASE 1: IFRAME MANIPULATION ===== driver.get("https://the-internet.herokuapp.com/iframe") driver.maximize_window() # Switch context into the isolated iframe document via frame element ID wait.until(EC.frame_to_be_available_and_switch_to_it((By.ID, "mce_0_ifr"))) # Target the nested contenteditable editor body editor = wait.until(EC.visibility_of_element_located((By.ID, "tinymce"))) editor.clear() editor.send_keys("Souvik Nandi Iframe Traversal Verified.") assert "Souvik Nandi" in editor.text, "Payload injection failed!" print(f"[IFRAME] Successfully updated internal frame content: '{editor.text}'") # Restore execution focus back to the top-level root DOM document driver.switch_to.default_content() # ===== PHASE 2: WINDOWS & TABS SWITCHING ===== driver.get("https://the-internet.herokuapp.com/windows") main_window_handle = driver.current_window_handle print(f"[ROOT] Main Window Handle: {main_window_handle}") # Trigger event opening a new browser tab driver.find_element(By.LINK_TEXT, "Click Here").click() wait.until(EC.number_of_windows_to_be(2)) # Identify and switch context to the newly spawned secondary tab new_window_handle = [h for h in driver.window_handles if h != main_window_handle][0] driver.switch_to.window(new_window_handle) # Validate tab title and child DOM payload child_title = driver.title assert child_title == "New Window", f"Title mismatch! Received: {child_title}" header_elem = driver.find_element(By.TAG_NAME, "h3") assert header_elem.text == "New Window" print(f"[CHILD] Verified Child Tab Title: '{child_title}'") # Terminate child tab and restore focus to primary parent handle driver.close() driver.switch_to.window(main_window_handle) assert driver.current_window_handle == main_window_handle print("[PASS] Child tab closed; primary window context restored.") finally: # Complete driver session teardown driver.quit()
```

EXECUTION VERIFICATION & EVALUATION

Output, Observations & Experimental Results

5. OUTPUT & TERMINAL EXECUTION TRACE

```
[INIT] ChromeDriver session active | Platform: W3C Compliant
[NAVIGATE] GET https://the-internet.herokuapp.com/iframe HTTP 200 OK
[SWITCH] EC.frame_to_be_available_and_switch_to_it(By.ID, 'mce_0_ifr') satisfied in 164ms
[ACTION] Located #tinymce body | Dispatched clear() and send_keys()
[IFRAME] Successfully updated internal frame content: 'Souvik Nandi Iframe Traversal Verified.'
[RESET] Invoked driver.switch_to.default_content() | Focus returned to root document
[NAVIGATE] GET https://the-internet.herokuapp.com/windows | Root Handle: 0F46E82782A...
[TRIGGER] Dispatched click() on link 'Click Here' | Target: _blank
[WAIT] EC.number_of_windows_to_be(2) satisfied | Handle Set: [0F46..., 9C1B...]
[SWITCH] Switched execution context to Child Handle: 9C1B53009C...
[ASSERTION] Child document.title == 'New Window' | Status: PASSED
[TEARDOWN_TAB] Executed driver.close() on Child Tab | Process count decremented to 1
[RESTORE] Invoked driver.switch_to.window(main_window_handle) | Active: 0F46E82782A...
[PASS] Child tab closed; primary window context restored. Exit code: 0
```

6. UI & CONTEXT STATE VERIFICATION MATRIX

Execution Lifecycle State Machine • Frame & Multi-Tab Context Traversal Verification

Step	Execution Phase	Context Target	Window Handle / Frame ID	Validation Status
01	Frame Navigation	Embedded Editor	iframe#mce_0_ifr	CONTEXT SWITCHED
02	Payload Injection	Body #tinymce	Text: "Souvik Nandi..."	VERIFIED (PASSED)
03	Root Restoration	Top-Level HTML	switch_to.default_content()	RESTORED
04	Tab Spawn Poll	Secondary Tab	len(window_handles) == 2	CONDITION MET
05	Child Assertion	Child Viewport	Title: "New Window"	ASSERTION PASSED
06	Tab Close & Revert	Primary Viewport	handle: main_window_handle	RESTORED & SAFE

7. IN-DEPTH TECHNICAL OBSERVATIONS & ENGINEERING ANALYSIS

- DOM Scope Sandboxing:** Browsers isolate `<iframe>` elements into discrete Window and Document objects. Attempting direct lookups on inner elements without calling `switch_to.frame()` causes WebDriver to search solely within the parent frame DOM, invariably resulting in false negative `NoSuchElementException` faults.
- Non-Deterministic Handle Sets:** The collection returned by `driver.window_handles` behaves as an unordered set. Production scripts must never assume the newest tab is index `[-1]`; filtering against known stored handles (e.g. `[h for h in handles if h != main]`) is mandatory for zero-flakiness.
- Session Teardown Discipline:** Closing a secondary tab using `driver.close()` terminates that specific viewport but leaves the WebDriver cursor pointing to an invalid, orphaned handle. Calling `driver.switch_to.window(main_window_handle)` immediately afterward is essential to restore valid communication with the browser.

8. CONCLUSION

This laboratory experiment demonstrated effective techniques for context switching across both internal DOM frame boundaries and OS-level browser window handles. By pairing explicit condition polling (`EC.frame_to_be_available_and_switch_to_it` and `EC.number_of_windows_to_be`) with disciplined context transitions, the automated workflow achieved complete deterministic reliability.

Laboratory Compliance Note: The test script strictly adheres to W3C WebDriver context management specifications. Process handles are deterministically tracked and closed, preventing background zombie browser processes.